Lab 1 Report:

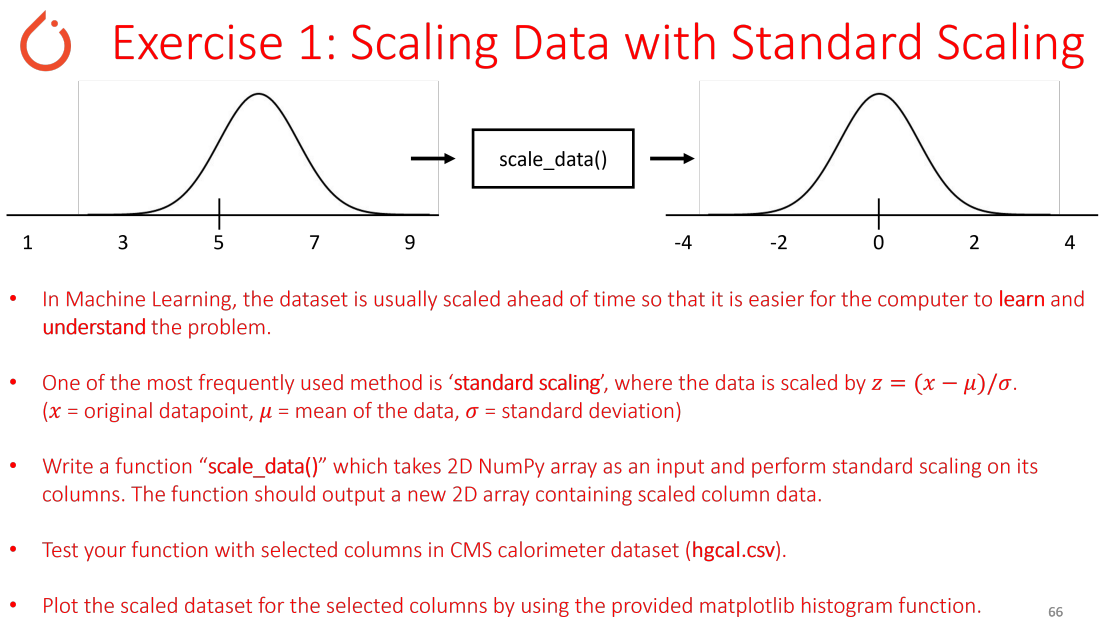
Data Preparation Techniques for Machine Learning

Nate Chaoweeraprasit

In [1]:
```
# Import necessary libraries

%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import Image # For displaying images in colab jup
```

In [2]:
```
Image('lab1_exercise1.PNG', width = 1000)
```

Out[2]:



In [3]:
```
# Load the dataset (.csv) using pandas package

CMS_calori_dataset = pd.read_csv('hgcal.csv')

# .head directive on the panda dataframe displays the first n-rows

CMS_calori_dataset.head(n = 10)
```

Out[3]:

	Unnamed: 0	x	y	z	eta	phi	ener
0	0	179.50383	-23.632137	-7.878280	-0.0435	-0.130900	0.2001
1	1	-143.63881	110.217940	-72.706795	-0.3915	2.487094	2.7345
2	2	179.50383	-23.632120	-146.429610	-0.7395	-0.130900	0.4239
3	3	-172.67310	54.443620	-238.065340	-1.0875	2.836160	0.7139
4	4	-180.88046	7.897389	-238.065340	-1.0875	3.097959	0.0000
5	5	-180.88045	-7.897438	-238.065340	-1.0875	-3.097959	0.0344
6	6	-152.69838	-97.279590	-265.020540	-1.1745	-2.574361	0.5801
7	7	-23.63213	179.503810	-325.172060	-1.3485	1.701696	0.4114
8	8	-152.69835	97.279594	89.977780	0.4785	2.574361	0.1831
9	9	-176.76110	39.187016	107.930240	0.5655	2.923426	0.3375

In [4]:

```
# Convert the panda dataframe into numpy 2D array

CMS_calori_dataset_np = CMS_calori_dataset.to_numpy()

# The converted numpy array has the dimension of 420 (rows) x 8 (colum

# check array dimensions
print(CMS_calori_dataset_np.shape)
```

```
(420, 8)
```

In [5]:

```
# Extract only x, y, z, eta, phi and energy columns from the dataset a
# Name this new 2D array CMS_calori_dataset_np_sub.
# The array should have dimension 420 (rows) x 6 (columns)

# by column, only want columns index 1 - 6:
# 1 - x
# 2 - y
# 3 - z
# 4 - eta
# 5 - phi
# 6 - energy

col_indices = [1, 2, 3, 4, 5, 6]
CMS_calori_dataset_np_sub = CMS_calori_dataset_np[:, col_indices]

# check array dimensions
print(CMS_calori_dataset_np_sub.shape)
```

```
(420, 6)
```

scaling requires finding the mean + stdev of each column, then doing the scaling

process to each index in each column

for each term in the column x_i, we do

$$x_{\text{scaled}} = \frac{x_i - \bar{x}}{\sigma}$$

In [6]:
```
# scaling function
def scale_data(arr):

    # exception gate; ensure array is 2D array
    if arr.ndim != 2:
        raise ValueError("array isn't 2D")

    # create copy so we don't affect original array
    scaled_data = arr.copy()

    # iterate by column
    for i in range(scaled_data.shape[1]):

        # for each column, find mean and stdev
        column_mean = np.mean(scaled_data[:, i])
        column_std = np.std(scaled_data[:, i])

        # then do (x-m)/std to find scaled value at each point
        scaled_data[:, i] = (scaled_data[:, i] - column_mean) / column

    return scaled_data
```

In [7]:
```
# Test the function with CMS_calori_dataset_np_sub
CMS_calori_dataset_np_sub_scaled = scale_data(CMS_calori_dataset_np_su

print('unscaled data:')
print(CMS_calori_dataset_np_sub)
print('\nscaled data:')
print(CMS_calori_dataset_np_sub_scaled)
```

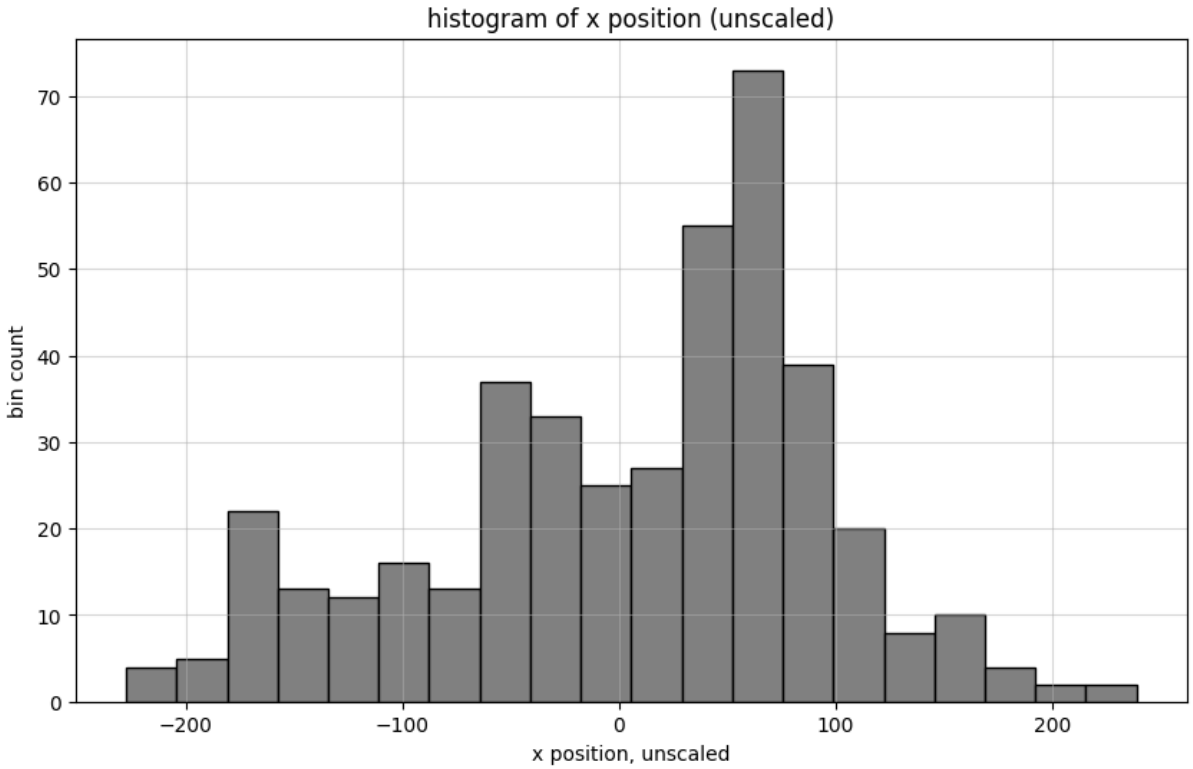
```
unscaled data:
[[ 1.7950383e+02 -2.3632137e+01 -7.8782797e+00 -4.3500002e-02
  -1.3089974e-01  2.0012644e-01]
 [-1.4363881e+02  1.1021794e+02 -7.2706795e+01 -3.9149997e-01
   2.4870942e+00  2.7345939e+00]
 [ 1.7950383e+02 -2.3632120e+01 -1.4642961e+02 -7.3950000e-01
  -1.3089965e-01  4.2390990e-01]
 ...
 [-3.5694687e+01 -5.0977300e+01  5.2279800e+02  2.8250000e+00
  -2.1816616e+00  5.3157926e-01]
 [-1.6106770e+01 -6.0111294e+01  5.2279800e+02  2.8250000e+00
  -1.8325956e+00  1.4225300e-01]
 [ 6.0111294e+01 -1.6106758e+01  5.2279800e+02  2.8250000e+00
  -2.6179916e-01  2.0927284e+00]]

scaled data:
[[ 1.91214438 -0.51027049 -0.44193343 -0.47341363 -0.31488841 -0.384103
07]
 [-1.64893594  1.11270897 -0.61977219 -0.64676625  1.28785592  0.998737
68]
 [ 1.91214438 -0.51027028 -0.82200995 -0.82011889 -0.31488835 -0.262003
69]
 ...
 [-0.45937553 -0.84184024  1.01382735  0.95550021 -1.57037143 -0.203257
79]
 [-0.24351375 -0.95259315  1.01382735  0.95550021 -1.3566721  -0.415679
62]
 [ 0.59642074 -0.41902261  1.01382735  0.95550021 -0.39502545  0.648526
92]]
```

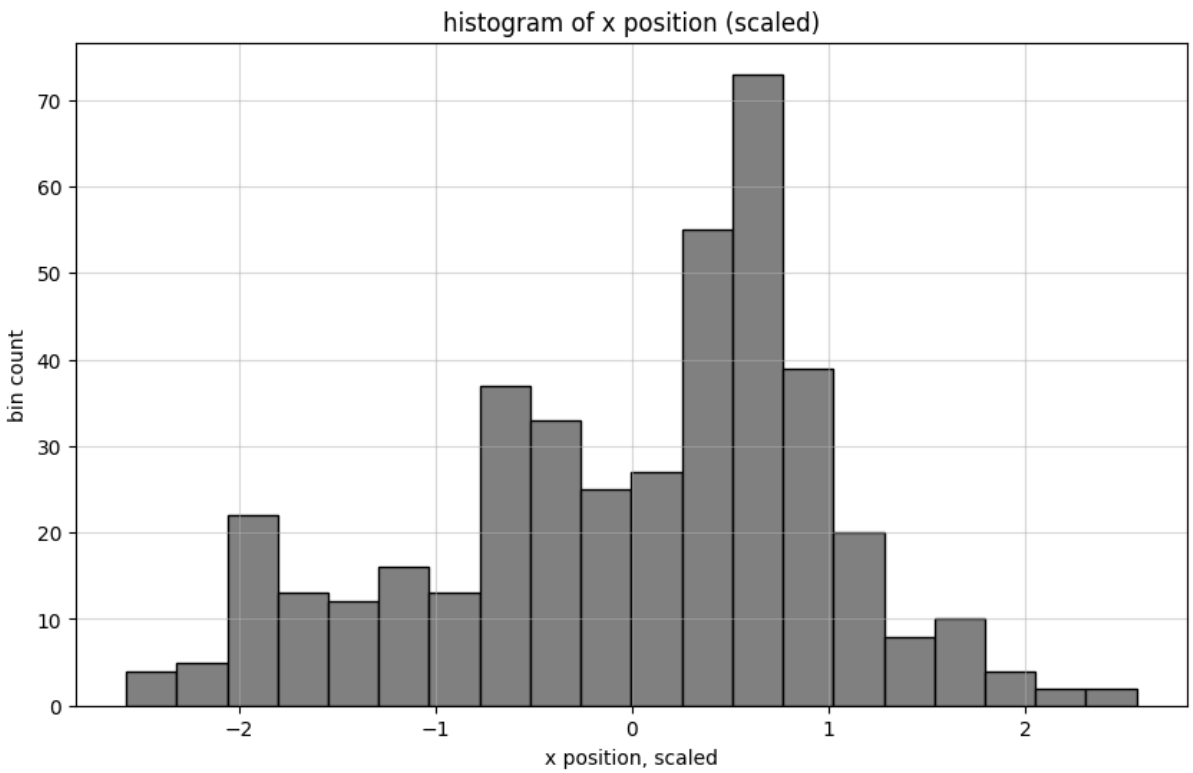
we can see that the scaled values are a lot more manageable ^^^

In [8]:

```
# Confirm the data is scaled for 'x' column

x_pos_col_index = 0

# plot unscaled x position data
plt.figure(figsize = (10, 6))
plt.grid(alpha = 0.5)
plt.hist(CMS_calori_dataset_np_sub[:, x_pos_col_index], bins = 20, fac
plt.xlabel('x position, unscaled')
plt.ylabel('bin count')
plt.title('histogram of x position (unscaled)')
plt.show()

# plot scaled x position data
plt.figure(figsize = (10, 6))
plt.grid(alpha = 0.5)
plt.hist(CMS_calori_dataset_np_sub_scaled[:, x_pos_col_index], bins =
plt.xlabel('x position, scaled')
plt.ylabel('bin count')
plt.title('histogram of x position (scaled)')
```

```
plt.show()
```

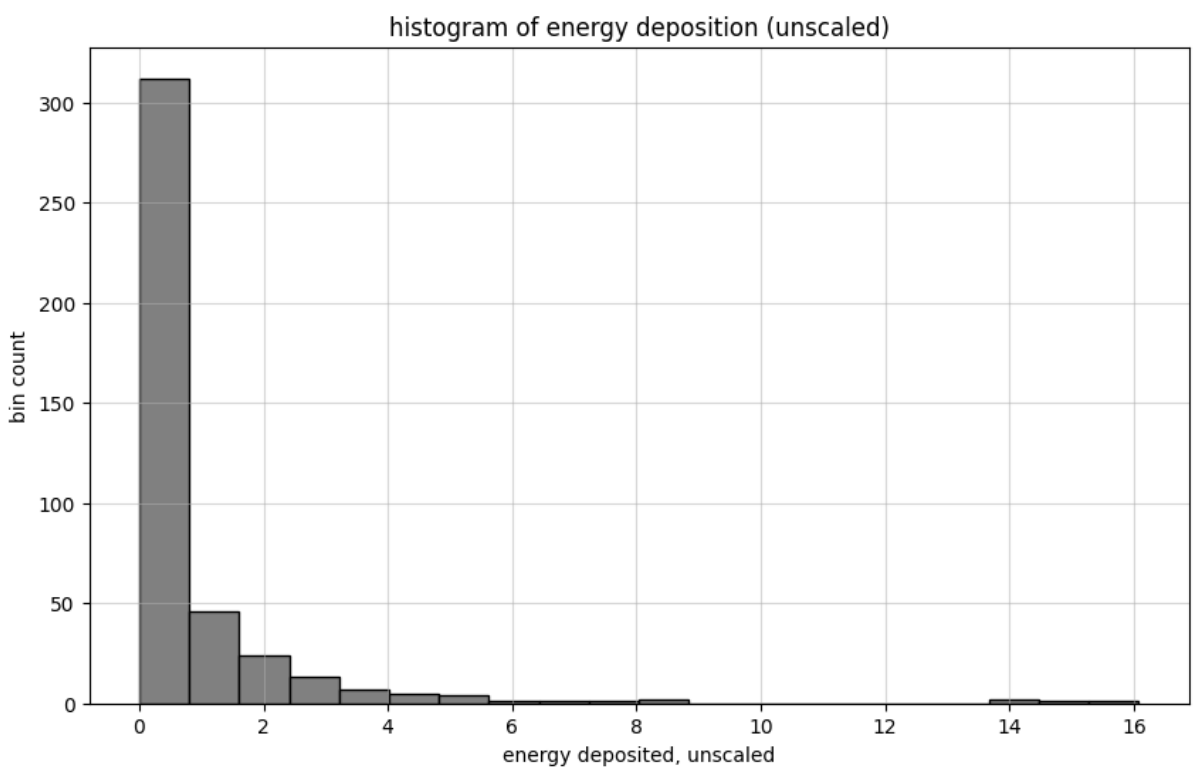


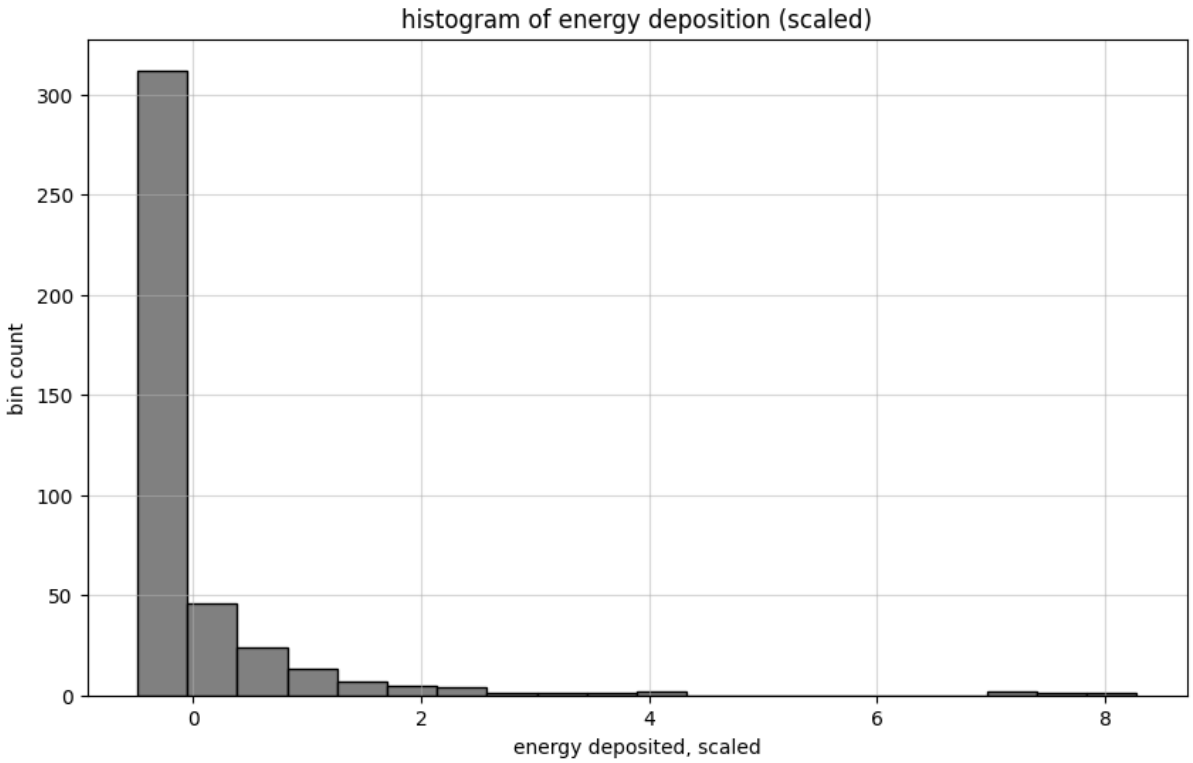


In the second scaled plot, we can see that the tick marks in the x position axis span from ~ -2 to +2, instead of -200 to 200. This indicates that our scaling function is effective.

In [9]:
```
# Confirm the data is scaled for 'energy' column
```

```
# repeat for energy deposition values

energy_col_index = 5

plt.figure(figsize = (10, 6))
plt.grid(alpha = 0.5)
plt.hist(CMS_calori_dataset_np_sub[:, energy_col_index], bins = 20, fa
plt.xlabel('energy deposited, unscaled')
plt.ylabel('bin count')
plt.title('histogram of energy deposition (unscaled)')
plt.show()

plt.figure(figsize = (10, 6))
plt.grid(alpha = 0.5)
plt.hist(CMS_calori_dataset_np_sub_scaled[:, energy_col_index], bins =
plt.xlabel('energy deposited, scaled')
plt.ylabel('bin count')
plt.title('histogram of energy deposition (scaled)')
plt.show()
```



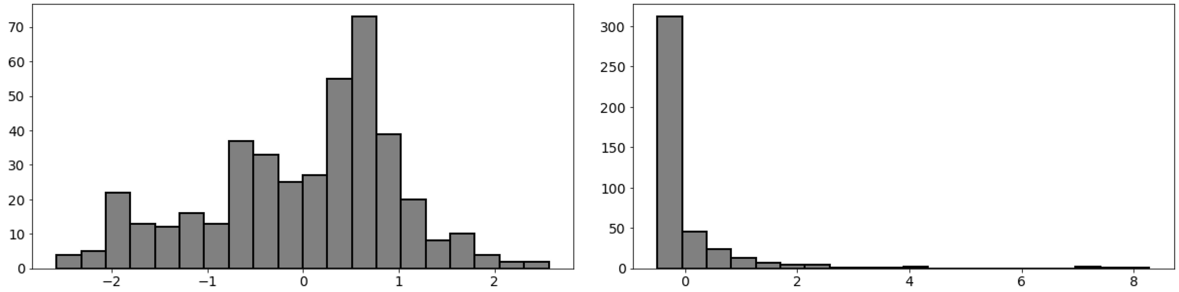


Same observation goes for the energy plot. Both match the answer plots below.

Expected histogram outputs - Feel free to style your plot differently

In [10]:
```
Image('lab1_e1_expected_outputs.PNG', width = 1000)
```

Out[10]:

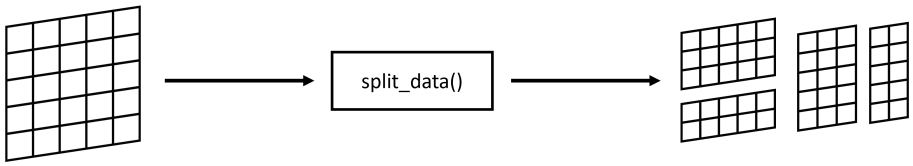


Scaled ‘x’ Scaled ‘energy’

In [11]:
```
Image('lab1_exercise2.PNG', width = 1000)
```

Out[11]:

Exercise 2: Data Splitting



- In this exercise you will write a function called **split_data**() which given a NumPy array, it splits the array into sub-arrays.
- Data splitting is used to divide the dataset into training, validation and testing sets, which we will describe in later lab.
- The function should take following parameters
  - arr – 2D NumPy array representing a dataset
  - split_proportions – a list containing split ratios, e.g., [0.2, 0.3, 0.5]
  - axis – a direction to be splitted (0 = row-wise, 1 = column-wise)
- Test your function on the scaled dataset from exercise 1 with given parameters in the lab template.
- Confirm that your sub arrays have correct dimensions by printing their shape

68

In [12]:

```
# Create the splitting function

def split_data(arr, split_proportions, axis):

    # exception gate; ensure values are valid, and that are normalized
    for proportion in split_proportions:
        if proportion < 0 or proportion > 1:
            raise ValueError('split proportions must be between 0 and
        if sum(split_proportions) != 1:
            raise ValueError('split proportions must sum to 1')

    # split array by proportion, regardless of how many proportions ha
    split_data_list = []
    start_index = 0

    # iterate on each proportion
    for proportion in split_proportions:

        # identify start/end index
        end_index = start_index + int(proportion * arr.shape[axis])

        # make sure we're splitting on right axis
        # filter for array data to within index range
        if axis == 0:
            split_data_list.append(arr[start_index:end_index, :])
        else:
            split_data_list.append(arr[:, start_index:end_index])
        start_index = end_index

    return split_data_list
```

In [13]:

```
# Test your split function against scaled CMS Calorimieter dataset fro
```

```
sub_data_list_1 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                                     split_proportions = [0.6, 0.2, 0.2
```

In [14]:
```
# Confirm that dataset has been split into correct shapes
# The correct dimensions should be (252, 6) (84, 6) (84, 6)

print(sub_data_list_1[0].shape, sub_data_list_1[1].shape, sub_data_lis
```

```
(252, 6) (84, 6) (84, 6)
```

In [15]:
```
# Test your split function against scaled CMS Calorimieter dataset fro

sub_data_list_2 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                                                split_proportions = [0
```

In [16]:
```
# Confirm that dataset has been split into correct shapes
# The correct dimensions should be (420, 3) (420, 3)

print(sub_data_list_2[0].shape, sub_data_list_2[1].shape)
```

```
(420, 3) (420, 3)
```